2 COMPUTATIONAL COMPLEXITY AND QUANTUM COMPUTING

In this chapter we introduce the basic concepts of quantum computing. We examine the reasons that push us to search for new computational paradigms and why quantum computing appears to be promising.

In order to fully understand the expectations placed on quantum computing, it is first necessary to introduce some elements of computability and complexity. We will, therefore, examine the foundations of computer science starting from computational models. We will give a definition of a computational problem, and see which problems we can solve with a computer. Finally, we will study the computational complexity of problems by classifying them according to the resources required to find a solution.

Having completed this introduction to computer science, we will see how quantum computers fit into this analysis. We will begin by presenting a computational model that allows us to define which problems are solvable on a quantum computer, and we will note how this naturally leads us toward quantum-based quantum computers. We will see which complexity classes can be defined considering this new paradigm, and we will try to understand what relationships exist between the classes of problems identified with the classical model of computation and those derived from the quantum model.

In the second section of this chapter we will present in a more rigorous way quantum gate-based computers. We will see what is meant by a quantum gate, present the most common gates, and construct a universal set of gates. We will then see what it means to program a quantum gate-based computer by presenting the Deutsch algorithm.

The chapter concludes by presenting *Quantum Adiabatic Computing* (QAC): an alternative paradigm to computation based on quantum gates. We will describe the theoretical foundations of this paradigm and show that QAC and quantum gate-based computing are equivalent. We will see that programming a machine that exploits the QAC paradigm means translating the problem we want to solve into a minimization problem solvable through *annealing*.

2.1 INTRODUCTION TO COMPUTER SCIENCE

Computer science deals with studying which problems can be solved through a computer and which resources are necessary to find the solution. We can divide these two objectives into two parallel and complementary lines of research. On one hand, to prove that a problem is solvable, it is sufficient to show an algorithm capable of solving that problem, without worrying too much about its complexity. On the other hand, to understand which re-

sources are necessary, we would like to find a lower bound on the operations that must be performed to obtain the solution.

These lines of research are complementary in the sense that, ideally, once a lower bound has been identified, we would like to be able to construct an algorithm that matches it. In practice, this is not always possible, because constructing algorithms that behave well on all instances of a problem is difficult, and because the lower bound of the required operations is not always known, so one must settle for the best algorithm developed so far.

We have referred to an algorithm intuitively knowing that it is a sequence of operations that allows us to find the answer to a problem. To formally study the characteristics of an algorithm, this intuition is not sufficient; we need something that we can characterize more precisely. We, therefore, introduce computational models.

2.1.1 Computational models

A computational model allows us to mathematically define the concept of an algorithm. These abstractions were first introduced by Turing and Church to find a solution to Hilbert's *entscheidungsproblem*[1]. We now consider two equivalent computational models: the *Turing machine* and the *circuit model*.

Turing machine

This is the most well-known computational model. Introduced by Turing to formally describe the notion of an algorithm that performs a computational task.

A Turing machine can be seen as the conceptualization of a computer. It is composed of four main elements:

PROGRAM: which allows the encoding of an algorithm;

FINITE STATE CONTROL: unit that acts as the processor;

TAPE: memory of the computer;

READ-WRITE TAPE-HEAD: interacts with the memory.

It is possible to define various versions of the Turing machine. For our discussion we can consider the simplest version, in which the memory tape is unbounded to the right and bounded to the left. The symbols that can be written on the tape are 0, 1, b (the blank symbol) and the special symbol $\triangleright$ that marks the left end of the tape. These symbols form our alphabet Γ: everything that can be represented on the tape consists of finite-length strings Γ^* made up of symbols from the alphabet (except $\triangleright$, which appears only at the beginning of the tape). The finite state control consists of a finite set of states $q1, \ldots, q_m$ plus two special states q_s and q_h that represent, respectively, the initial state of the machine and the final (halt) state; the latter is reached when the machine has finished the computation.

A program for the Turing machine consists of program lines; each line is a quintuple $\langle q, x, q', x', s \rangle$ which is interpreted as follows: if the machine is

1 For every mathematical problem, does there exist an algorithm capable of solving it?

in state q and reads the symbol $x \in \Gamma$ on the tape, then it transitions to state q', writes the symbol $x' \in \Gamma$ on the tape, and moves as indicated by s (to the right $+1$, to the left -1, or remains stationary 0).

Although the Turing machine model may appear simple, it is capable of computing a very large number of functions; in particular, this model completely captures the notion of computing a function using an algorithm. This thesis was proposed independently by Church and Turing and is known as the *Church–Turing thesis*:

Thesis 1. *The class of functions computable by a Turing machine corresponds exactly to the class of functions which we would naturally regard as being computable by an algorithm.*

Church–Turing thesis states that the Turing machine is a computational model capable of providing a rigorous definition of the intuitive concept of an algorithm, and is therefore suitable as a foundation for computer science.

The Turing machine we have described can be modified, for example, by imagining having multiple memory tapes; this new machine could be able to solve problems more quickly than the original, but we will always be able to simulate the multi-tape machine with a single-tape machine. This means that considering multiple tapes does not increase the power of the Turing machine.

Another modification that we can make to this computational model, which will be useful later, is to consider a probabilistic version of the Turing machine: if the machine is in state q and the symbol x is read, a coin is flipped; if the result is heads, the machine transitions to state q_H, otherwise to state q_T. This modification can also be simulated with a Turing machine that explores all possible computation paths one at a time (using exponentially more steps), so once again the set of computable functions is not extended.

UNDECIDABILITY: Computational models, and with them computer science, were developed to answer Hilbert's entscheidungsproblem. Church, with the λ-calculus[2], and Turing, with the Turing machine, showed that the answer is no: there cannot exist an algorithm to decide all the problems of mathematics.

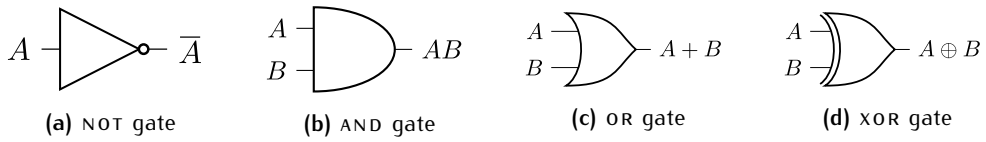
This result clearly divides mathematical problems into two classes: those that can be solved with an algorithm and those so difficult as to be computationally unsolvable. We call the problems belonging to this class *undecidable*. Turing shows that the *halting problem* (the problem of determining whether a Turing machine halts given an input) is, in general, undecidable[3]. In addition to the problems used in the proofs of Church and Turing, there exist many other undecidable problems, for example finding a homomorphism between two topological spaces or some problems related to the behavior of dynamical systems.

2 Another computational model equivalent to the Turing machine.

3 Church shows that there is no computable function that allows one to determine whether two λ-calculus expressions are equivalent.

Circuit model

The circuit model is an alternative model of computation in which we describe an algorithm by implementing it through a circuit made of logic gates. This approach is interesting because it brings us closer to the first quantum model of computation: the gate-based model.



(a) NOT gate **(b)** AND gate **(c)** OR gate **(d)** XOR gate

Figure 2.1: Logic gate

In Figure 2.1 the most well-known logic gates are shown, with which we can build a digital circuit. To these we normally add the possibility of obtaining a copy of a bit (a wire that splits), an operation we call FANOUT, and the possibility of swapping two bits, called CROSSOVER.

It is possible to show that with this limited set of gates one can compute any function $f : \{0,1\}^n \to \{0,1\}^m$. This is not sufficient to describe a useful computational model. Before seeing which characteristics we require, we define the concept of a circuit family.

A *family of circuits* is a set of circuits $\{C_n\}$ indexed by $n \in \mathbb{N}$. The circuit C_n has n input bits and a finite number of extra work bits and output bits. Given a circuit C_n and a bit string x of maximum length n as input of the circuit, we denote by $C_n(x)$ the output of the circuit.

We say that a family of circuits is *consistent* if all circuits in the family compute the same function $C(\cdot)$. Formally, the definition of consistency is: the family of circuits $\{C_n\}$ is consistent if for $m < n$ and x of length at most m bits, then $C_m(x) = C_n(x)$.

Finally, we define a family of circuits to be *uniform* if there exists an algorithm for a Turing machine that, given n as input, is able to generate a description of C_n. That is, this algorithm must provide the list of gates that constitute C_n, and the description of how these gates are connected to each other.

We are finally ready to describe the formal characteristics of the circuit model of computation. In this model we consider functions computable by families of circuits $\{C_n\}$ such that:

- the circuits in the family are *acyclic*: no loops must be present in the circuit, this prevents the occurrence of possible instabilities;
- $\{C_n\}$ is a *consistent* family of circuits: all circuits compute the same function (on inputs of increasing size);
- $\{C_n\}$ is a *uniform* family of circuits: the circuits can be generated from a reasonable algorithm.

It is possible to show that the class of functions computable by a consistent, uniform, and acyclic family of circuits is exactly the same as that computable on a Turing machine.

2.1.2 Computational problems

The theoretical models that we have defined in the previous section (and other equivalent ones that have been proposed in the course of studying computer science) allow us to formally study computational problems. This analysis depends on the answer to three fundamental questions:

WHAT IS A COMPUTATIONAL PROBLEM: by computational problem we mean a specific class of problems called *decision problems*. This choice allows us to isolate a restricted class to simplify the analysis and make progress in the development of the theory. We will see that the results obtained extend well beyond decision problems;

HOW TO DEVELOP AN ALGORITHM TO SOLVE IT: once the problem has been identified, it is necessary to develop an algorithm capable of finding a solution. We are then interested in determining whether there are general algorithms capable of solving large classes of problems. Finally, we would like to find tools to guarantee that an algorithm actually works as expected;

WHAT RESOURCES ARE NECESSARY FOR THE SOLUTION: an algorithm consumes computational resources such as time, space, and energy; we want to know whether the developed algorithm consumes more resources than actually necessary. Moreover, we would like to classify problems according to the resources required to solve them.

Computational complexity

The study of computational complexity aims to quantify the computational resources required to solve a given problem. To study resource consumption independently of the characteristics of the reference architecture[4], *asymptotic notation* is introduced, which captures the essential behavior of a function:

- O ("big O"): used to define the *upper bound* of a function;
- Ω ("big Omega"): used to define the *lower bound* of a function;
- Θ ("big Theta"): is the combination of O and Ω, describing the asymptotic behavior of a function.

Computational complexity considers the spatial and temporal resources required to solve a problem and seeks to demonstrate the existence of a lower bound for these resources whether or not an optimal algorithm exists that meets this lower bound. Clearly, the development of algorithms should aim to find an algorithm that does not use more resources than necessary, that is, the theoretical lower bound identified by the study of computational complexity.

In developing the theory of computational complexity, it has been observed that resource consumption depends on the computational model of reference; for example, the three different Turing machines presented in the previous section can solve the same problem with a very different number of operations.

4 We mean independence from specific architectural decisions that, for example, allow a processor to save a few clock cycles whenever a certain operation is performed. We will soon see that different computational models can, instead, have asymptotically different resource consumption.

This problem has been clearly addressed by dividing problems into two classes:

- *easy tractable* or *feasible* problems: are problems for which there exists an algorithm capable of finding a solution in *polynomial* time with respect to the size of the problem. An algorithm whose resource consumption grows as $O(p(n))$, where n is the size of the problem and $p(\cdot)$ is a polynomial, is called *efficient*;
- *hard*, *intractable*, or *infeasible* problems: if the best algorithm requires exponential resources[5]. In this case we speak of *inefficient* algorithms.

There are several reasons why the choice of dividing problems into two classes of computational complexity (polynomial and exponential) is appropriate. The first is historical: the polynomials describing most proposed algorithms do not have high degree, which means polynomial algorithms almost always outperform exponential algorithms. The second reason is more foundational and is due to the *strong Church–Turing thesis*:

Thesis 2. *Any model of computation can be simulated on a probabilistic Turing machine with at most a polynomial increase in the number of elementary operations required.*

This means that probabilistic Turing machines appear to be the strongest "reasonable" model of computation. Formally, we can state that if, considering a certain computational model (different from the probabilistic Turing machine), it is possible to solve a problem with k elementary operations, then it is possible to solve the same problem on a probabilistic Turing machine with at most $p(k)$ elementary operations (where $p(\cdot)$ is a polynomial).

Reversing this description, the strong Church–Turing thesis states that, given a certain problem to be solved, if there is no efficient algorithm for the probabilistic Turing machine, then an efficient algorithm does not exist for any computational device.

Classes of problems

For now we have divided problems into polynomial and exponential, let us now consider finer divisions of computational problems, taking into account not only the time required but also the space needed to find the solution. Before dividing problems into classes, we formally define the decision problems introduced at the beginning of this section.

A decision problem is a problem whose answer is yes or no. We can define them formally using the terminology of *formal languages*. A language L over the alphabet Γ is a subset of the set Γ^* of all (finite) strings of symbols from Γ. A string represents the input of our problem and can be written on the tape of a Turing machine. The machine performs a computation that may terminate by writing on the tape a string equivalent to yes or no depending on whether the input string belongs to the language L or not.

In general, a language L is *decided* by a Turing machine if the machine is able to decide whether an input x on its tape is a member of the language L or not.

5 We consider exponential anything that grows faster than a polynomial; $n^{\log n}$, with some abuse of notation, is considered an exponential function.

P CLASS: We say that a problem can be solved in polynomial time if there exists a Turing machine capable of deciding whether the input x of size n belongs to the language in time $O(n^k)$ for some finite k; we say that this problem belongs to $\mathsf{TIME}(n^k)$. The set of all languages in $\mathsf{TIME}(n^k)$ is denoted by P. P is our first example of a *complexity class*.

To prove that a language L belongs to P, it is sufficient to show an algorithm that decides L in polynomial time. Unfortunately, there exist problems for which such an algorithm has not yet been found. One may then ask whether there exist problems outside P.

Conversely, although there exist non-constructive proofs showing that there must exist problems that require exponential resources, it seems very difficult to prove that a specific problem does not belong to P.

NP CLASS: There exist problems for which a polynomial algorithm has not been found, but if the solution to the problem is somehow provided, it is possible to apply an efficient algorithm to verify such a solution. Problems with this property belong to the class NP. Formally, the set NP contains the languages L such that there exists a Turing machine M such that:

- if $x \in L$, there exists a witness string w such that M halts answering "yes" after a time polynomial in the size of x when the machine starts from the state xbw (b is the blank space);
- if $x \notin L$ for any witness string w, M halts answering "no" after a time polynomial in the size of x when the machine starts from the state xbw.

The most famous open problem in computer science is whether or not there are problems in NP which are not in P, often abbreviated as the $\mathsf{P} \neq \mathsf{NP}$ problem.

Within the class of problems NP we find the NP-*complete* problems. A language L is NP-complete if any other language in NP can be *reduced* to L. A language B is said to be reducible to a language A if there exists a Turing machine that, in polynomial time, starting from x produces $R(x)$ and $x \in B$ if and only if $R(x) \in A$. We can view reduction as a translation of one problem into another equivalent one: finding the solution to one means solving the other as well. NP-*complete* problems represent the hardest problems in the class NP, in the sense that finding a lower bound for an NP-*complete* problem means finding a lower bound for all problems in NP.

An NP-*complete* problem is the following: given a circuit with n input bits and one output bit, determine whether there exists an assignment of the input bits such that the output of the circuit is 1. This problem is called the *circuit satisfiability* problem or CSAT. The proof of NP-*completeness* of CSAT is due to the *Cook–Levin theorem*

Theorem 4. *CSAT is NP-*complete.

Once a language L that is NP-*complete* has been found, to prove that another language L' is NP-*complete* it is sufficient to find a reduction from L to L'.

PSPACE CLASS: If we stop considering the time required to solve a problem and instead focus on space consumption, we can define the class of problems PSPACE. This class contains the languages L that can be recognized by a Turing

machine using a polynomial number of working bits without time limitations. The classes of problems **P** and **NP** are subsets of **PSPACE**, but there are no proofs that these subsets are strict or not.

A CHAIN OF INCLUSIONS: We can organize the main classes of computational problems in the following chain of inclusions:

$$\mathsf{L} \subseteq \mathsf{P} \subseteq \mathsf{NP} \subseteq \mathsf{PSPACE} \subseteq \mathsf{EXP}. \tag{2.1}$$

Where **L** are the problems solvable in logarithmic space and **EXP** the problems solvable in exponential time.

Although most computer scientists believe that each of these inclusions is strict, there are no proofs of this. However, there exist results that allow us to state with certainty that there is not a single class of problems. The *time hierarchy theorem* and the *space hierarchy theorem* ensure respectively that $\mathsf{L} \subset \mathsf{PSPACE}$ and $\mathsf{P} \subset \mathsf{EXP}$. This means that in Equation 2.1 there is at least one strict inclusion, even if we do not know which one.

Approximate solutions

If we need to solve an **NP**-*complete* problem, today we can rely on many algorithms that perform well on most instances of these difficult problems. If we happen to be particularly unlucky and the instance we are trying to solve does not appear to have favorable properties for our algorithms, we can change approach and search for an *approximate solution*.

There exist **NP**-*complete* problems for which approximate algorithms are known that guarantee a certain maximum error with respect to the correct solution. There is an entire branch of computer science devoted to approximation algorithms. It is possible to define the class of languages **MAXSNP**, analogous to the class **NP**, for which it is possible to efficiently verify an approximate solution to the problem.

BPP CLASS: This is the last class of problems that we introduce considering classical computational models, and it will be useful in the next section where we will introduce quantum models of computation. To define this class of languages we change the computational model; we have already introduced the probabilistic Turing machine and have seen that it is the most powerful classical model of computation.

With this model we can define the class **BPP**, that is *bounded-error probabilistic time*, which contains the languages L for which there exists a probabilistic Turing machine M such that if $x \in L$ then M accepts x with probability $\geqslant \frac{3}{4}$, and if $x \notin L$ then M rejects x with probability $\geqslant \frac{3}{4}$[6].

The *Chernoff bound* implies that with a limited number of repetitions of the algorithm, the probability of success is amplified to the point of being essentially one. For this reason, languages in **BPP**, even more than those in **P**, are considered those that can be solved efficiently on a classical computer.

6 The value $\frac{3}{4}$ is arbitrary; it is sufficient that the probability is greater than $\frac{1}{2}$.

2.1.3 Quantum computational complexity

We now extend the classification of computational problems by considering a new computational model: *quantum circuits*. In Section 2.2.2 we will precisely define what a *quantum gate* is and how they can be assembled to create quantum circuits. For now, it is sufficient to know that a quantum gate applies a unitary transformation (introduced in Section 1.4.2) to one or more quantum bits (*qubits*).

BPQ Class

Thanks to the concept of quantum circuits we can define the class of problems **BQP**: the set of decision problems that can be solved with bounded probability of error using a polynomial size quantum circuit. More formally, we can say that the language $L \in$ **BQP** if there is a family of polynomial size quantum circuits which decides the language, accepting strings in the language with probability at least $\frac{3}{4}$, and rejecting strings which aren't in the language with probability at least $\frac{3}{4}$.

In this definition we can again see the concept of a family of circuits, this is because a quantum circuit can find the solution to a problem only up to a certain size; if this increases it will be necessary to modify the circuit. For a problem to be in **BQP** we require that:

- the size of the circuits in the family that solve the problem grows, at most, polynomially with respect to the size of the input string x;
- the family of circuits is *uniform*, that is, there exists a set of clear instructions, our algorithm, that generates the quantum circuit.

Relation with classical classes

One of the most significant results in quantum computational complexity is that **BQP** $\in$ **PSPACE**. It is clear that **BPP** $\in$ **BQP**. The question if **BPP** $\neq$ **BQP** remains open; in other words, a proof that a quantum computer is more powerful than a classical computer has not yet been found.

The inequality **BPP** $\neq$ **BQP** does not only affect the comparison between classical and quantum models of computation, but would also have repercussions on the hierarchy of classical inclusions. If we rewrite Equation 2.1 in light of **BQP** $\in$ **PSPACE** we obtain:

$$\mathbf{L} \subseteq \mathbf{P} \subseteq \mathbf{NP} \subseteq \mathbf{BQP} \subseteq \mathbf{PSPACE} \subseteq \mathbf{EXP}. \tag{2.2}$$

Assuming that **BPP** $\neq$ **BQP**, we could deduce that **BPP** $\neq$ **PSACE**.

There are strong doubts that a quantum computational model can solve all problems in **NP**, in particular **NP**-complete problems would be excluded. However, there are algorithms that suggest that a quantum computer can solve in polynomial time a problem outside **P** and **BPP**.

The class of problems that belong to **NP** but not to **NP**-complete are called **NPI** (**NP** *intermediate*) . The problem of prime factorization belongs to this class of problems and is solved by Shor's algorithm in polynomial time. This algorithm would seem to demonstrate the computational superiority of the quantum model; however, unfortunately, there is no proof that **P** $\neq$ **NPI**. This is the class of

problems on which researchers in quantum computation and quantum information focus to show the superiority of quantum computation over classical computation.

2.2 GATE BASED PARADIGM

The quantum gate-based computation paradigm is the direct implementation of the computational model that we have just described. This approach to quantum computation is founded on extensive theoretical work, part of which we have mentioned in the previous section, and for this reason most of the theoretical results on quantum computation derive from this paradigm.

As suggested by the name, this model is based on quantum gates; quantum gates manipulate *qubits* by applying physical transformations to them. The purpose of Chapter 1 was to convince us that these gates must have certain characteristics in order to comply with the laws of physics.

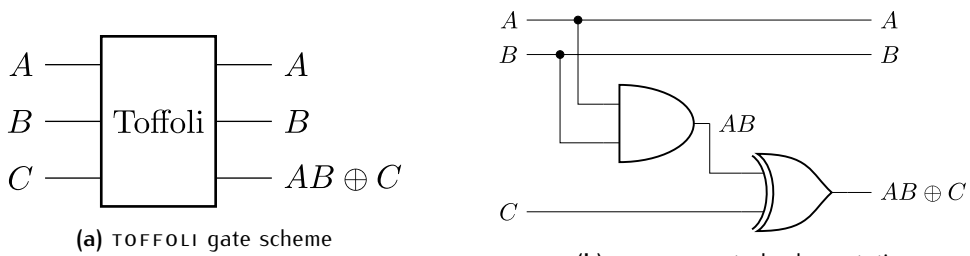
Quantum gates modify *qubits* by applying unitary operators. In Section 1.4.2 we focused on the physical meaning of a unitary operator and we saw how, knowing the state of the system after the application of the operator, we could infer the initial state.

We now focus on the implications that a unitary operator has on a computational model. The first consequence, the most obvious one, is that we are allowed to construct only reversible gates, that is, gates that allow us to determine the state of the input *qubits* if we know the state of the output *qubits*.

2.2.1 Classical reversible gates

Reversible gates do not exist only when dealing with quantum computers. There are numerous examples of classical reversible gates and there are studies that start from a completely reversible classical computation to drastically reduce the energy consumption of computers. It is also useful for us to examine some classical reversible gates because some considerations we can make about them will be useful in the study of quantum gates.

The simplest reversible gate is the NOT gate (Figure 2.1a). Another important classical reversible gate is the TOFFOLI gate shown in Figure 2.2[7].



(a) TOFFOLI gate scheme

(b) TOFFOLI gate Implementation

Figure 2.2: Circuit sampling

The TOFFOLI gate is also called *controlled controlled* NOT. From Figure 2.2 we see that the value of bit C is negated if both the control bit A and the control bit B are set.

7 Figure 2.2b shows the implementation of the TOFFOLI gate using the gates shown in Figure 2.1.

If we observe the truth table of the TOFFOLI gate (Figure 2.3) we can notice how the inputs are uniquely identified by the outputs (definition of a reversible gate). The TOFFOLI gate has three input bits, therefore the truth table has $2^3 = 8$ rows; since the outputs must be unique keys there are no repetitions in the outputs. This means that all possible combinations of three bits also appear in the outputs. We can then deduce that the TOFFOLI gate acts by *permuting* the input rows of the truth table. This is true in general:

A	B	C	A	B	$AB \oplus C$
0	0	0	0	0	0
0	0	1	0	0	1
0	1	0	0	1	0
0	1	1	0	1	1
1	0	0	1	0	0
1	0	1	1	0	1
1	1	0	1	1	1
1	1	1	1	1	0

Figure 2.3: TOFFOLI gate truth table

Theorem 5. *The output column of the truth table of a classical reversible gate is a permutation of the input column.*

This concept of permutation of the input will also reappear when we discuss quantum gates.

2.2.2 Quantum gate

In Chapter 1 we saw that spin is an example of a binary property, and we classified as *qubit* all those physical properties that, when measured, return only two values. We measured spin with respect to the coordinate axes and identified different spin states (*up* and *down*, *left* and *right*, *in* and *out*).

These states slightly change notation when we move from the study of physics to computer science. Before describing quantum gates we introduce this alternative notation for the states of a *qubit* and show a graphical representation that will be useful to understand the effect of quantum gates.

qubits

In computer science the states *up* and *down* (Equations 1.4 and 1.5) represent respectively a *qubit* off $|0\rangle$ and on $|1\rangle$. We denote with $|+\rangle$ and $|-\rangle$ the *qubits* in the states *right* and *left* (Equations 1.6 and 1.7); finally the states *in* and *out* (Equations 1.8 and 1.9) are represented by $|i\rangle$ and $|-i\rangle$. We can rewrite the cited equations considering the new notation:

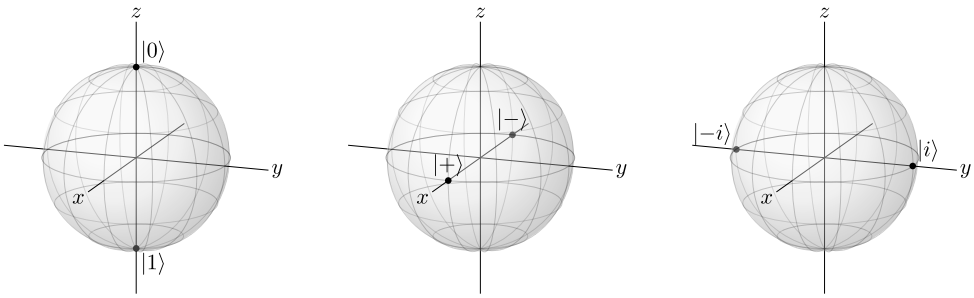
$$|+\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \qquad |-\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) \tag{2.3}$$

and:

$$|i\rangle = \frac{1}{\sqrt{2}}(|0\rangle + i\,|1\rangle) \qquad |-i\rangle = \frac{1}{\sqrt{2}}(|0\rangle - i\,|1\rangle) \tag{2.4}$$

To represent *qubits* in a graphical way that respects the mathematical properties of quantum states, we can use a sphere called the *Bloch sphere*. The sphere has radius 1 and every point on the sphere represents a *qubit*[8]. Figure 2.4 shows, on the Bloch sphere, the *qubits* described above.

8 Up to a global phase, which is however irrelevant.



(a) States $|0\rangle$ and $|1\rangle$ **(b)** States $|+\rangle$ and $|-\rangle$ **(c)** States $|i\rangle$ and $|-i\rangle$

Figure 2.4: Bloch Sphere **[bibid]**

Quantum gate

Recalling the results achieved in Chapter 1 and also considering what we have said about classical reversible gates in Section 2.2.1, we can make the following statements regarding quantum gates:

- quantum gates are linear maps preserving the total probability equal to 1[9];
- we can represent a quantum gate as a matrix that preserves the total probability equal to 1;
- quantum gates are unitary matrices and unitary matrices are quantum gates;
- a quantum gate $\mathbf{U}$ is reversible and $\mathbf{U}^\dagger$ is the inverse gate;
- classical reversible gates are valid quantum gates[10].

Now that we have described the main properties of quantum gates we can examine the effect that these gates have on *qubits*; we begin our analysis with gates that operate on a single *qubit*.

SINGLE QUBIT QUANTUM GATE: Quantum gates that operate on a single *qubit* can be visualized as *rotations* on the Bloch sphere. Any rotation can be traversed both forward and backward, which makes rotations reversible and, consequently, valid quantum gates. We now examine some rotations that define particularly interesting gates.

Rotations of 180° with respect to the axes define the *Paoli gates*:

- Paoli gate X: rotation with respect to the x axis. This gate is the quantum version of the classical NOT gate:

$$X|0\rangle = |1\rangle \qquad X|1\rangle = |0\rangle\,;$$

- Paoli gate Y: rotation with respect to the y axis. This gate has no classical counterpart and, applied to the *qubits* $|0\rangle$ and $|1\rangle$, modifies them as follows:

$$Y|0\rangle = i\,|1\rangle \qquad Y|1\rangle = -i\,|0\rangle\,;$$

9 Recall that the total probability of the state $|s\rangle = c_1\,|0\rangle + c_2\,|1\rangle$ formed by a *qubit* is $\alpha^2 + \beta^2$.

10 In particular, it is possible to show that a classical reversible gate permutes probability amplitudes, thus preserving the total probability equal to 1.

- Paoli gate Z: rotation with respect to the z axis. This is also a purely quantum gate, which acts as follows:

$$Z\,|0\rangle = |0\rangle \qquad Z\,|1\rangle = -\,|1\rangle\,;$$

- Paoli gate I: zero rotation. This gate does not modify the state of the *qubits*:

$$I\,|0\rangle = |0\rangle \qquad I\,|1\rangle = |1\rangle\,.$$

It is also possible to define these gates as matrices; the definition is the same as that of the operators that represented the observables σ_x, σ_y and σ_z (respectively Equations 1.3.3, 1.3.3 and 1.3.3) plus the identity matrix for the I gate. In fact, if we observe the matrices that describe the observables we note that, in addition to being Hermitian operators (which describe observables), they are also unitary, therefore valid quantum gates.

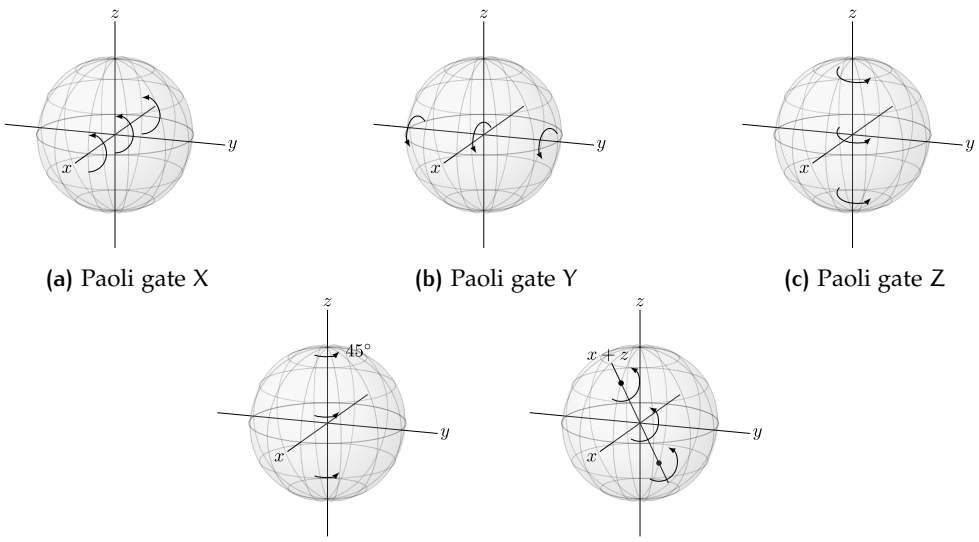
We can then consider rotations smaller than 180°; we define the S gate or *phase gate* as a rotation of 90° with respect to the z axis and by halving the rotation angle again we obtain the T gate (rotation of 45° with respect to z). The action of T on $|0\rangle$ and $|1\rangle$ is:

$$T\,|0\rangle = |0\rangle \qquad T\,|1\rangle = e^{i\frac{\pi}{4}}\,|1\rangle\,.$$

The last gate that acts on a single *qubit* that we consider is the *Hadamard gate* (H). This gate can be interpreted as a rotation of 180° with respect to the $x+z$ axis. H allows the creation of superposition of states[11], in particular:

$$H\,|0\rangle = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) = |+\rangle \qquad H\,|1\rangle = \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) = |-\rangle\,.$$

Figure 2.5 shows the described gates as rotations on the Bloch sphere.



(a) Paoli gate X **(b)** Paoli gate Y **(c)** Paoli gate Z

(d) Gate T **(e)** Hadamard gate H

Figure 2.5: Gate as rotation **[bibid]**

11 We will return to this in Section 2.2.2.

MULTIPLE QUBITS QUANTUM GATE: When we describe states composed of multiple *qubits* we must represent them as a *tensor product* $\otimes$[12]. For example, a state in which two *qubits* are both in the state $|0\rangle$ is written:

$$|0\rangle \otimes |0\rangle \text{ abbreviated as } |0\rangle\,|0\rangle \text{ abbreviated as } |00\rangle\,.$$

With two *qubits* the z basis is $\{\,|00\rangle\,, |01\rangle\,, |10\rangle\,, |11\rangle\,\}$. A generic state $|s\rangle$ will be the superposition of the basis states

$$|s\rangle = c_1\,|00\rangle + c_2\,|01\rangle + c_3\,|10\rangle + c_4\,|11\rangle\,,$$

and the total probability will be $|c_1|^2 + |c_2|^2 + |c_3|^2 + |c_4|^2$ which, if the state is normalized, will be equal to 1.

Some composite states can be written as a tensor product of single *qubit* states, for example:

$$\frac{1}{2}(|00\rangle - |01\rangle + |10\rangle - |11\rangle) = \frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \otimes \frac{1}{\sqrt{2}}(|0\rangle - |1\rangle) = |+\rangle\,|-\rangle\,.$$

These states are called *product states* or *simply separable states*. There exist states that cannot be decomposed into simple states. In this case we speak of *entangled states*, states in which complete knowledge of the state vector of the entire system does not provide any information about the states of the elements composing the system[13].

We study multiple qubits quantum gates precisely for their ability to create entangled states. The first gate that enables entanglement between *qubits* is the CNOT or CX.

The CNOT gate (*controlled* NOT) is a two-*qubit* gate; the first is called the *control*, the second is the *target qubit*. If the control *qubit* is set, the target *qubit* is flipped (hence the name controlled NOT). The behavior of the CNOT gate can be described as:

$$\text{CNOT}\,|a\rangle\,|b\rangle = |a\rangle\,|a \oplus b\rangle\,.$$

$$\text{CNOT} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 1 & 0 \end{pmatrix}$$

Figure 2.6: CNOT gate

Figure 2.6 reports the matrix definition of the CNOT. This gate is able to create entangled states. In particular:

$$\begin{aligned}
\text{CNOT}\,|+\rangle\,|0\rangle &= \frac{1}{\sqrt{2}}(|00\rangle + |11\rangle) = \left|\Psi^+\right\rangle \\
\text{CNOT}\,|-\rangle\,|0\rangle &= \frac{1}{\sqrt{2}}(|00\rangle - |11\rangle) = \left|\Psi^-\right\rangle \\
\text{CNOT}\,|+\rangle\,|1\rangle &= \frac{1}{\sqrt{2}}(|01\rangle + |10\rangle) = \left|\Phi^+\right\rangle \\
\text{CNOT}\,|+\rangle\,|1\rangle &= \frac{1}{\sqrt{2}}(|01\rangle - |10\rangle) = \left|\Phi^-\right\rangle\,.
\end{aligned}$$

12 Describing mathematically the tensor product is beyond the goal of this work.

13 Entanglement is one of the most interesting aspects of quantum mechanics. A detailed discussion of its physical implications is beyond the scope of this work. The reader is referred to [**bibid**].

The states $|\Psi^+\rangle$, $|\Psi^-\rangle$, $|\Phi^+\rangle$ and $|\Phi^-\rangle$ cannot be obtained as tensor products of single *qubit* states and are, therefore, entangled states. These four states are called *Bell states*; they are orthogonal and form a basis called the *Bell basis*.

Other important quantum gates are those we have already encountered when discussing classical gates; we have already seen that a classical reversible gate is a valid quantum gate. The gates of most interest to us are the TOFFOLI gate and the CROSSOVER gate.

It is not possible, however, to construct a quantum analogue of the FANOUT gate; therefore we cannot obtain a copy of a *qubit* without knowing its state. Measuring the state, however, would inevitably modify it, so in general we cannot duplicate a *qubit*. This is an important theoretical result and is known as the *non-cloning theorem*:

Theorem 6. *It is impossible to make a copy of an unknown quantum state. Qubits cannot be copied.*

Universal quantum gates

Just as there exist sets of classical gates that allow the construction of any other gate[14], we can identify sets of universal quantum gates. A set of universal quantum gates must be able to:

- create superpositions of states such as the Hadamard gate H;
- create entangled states such as the CNOT gate;
- create complex probability amplitudes; H and CNOT contain only real values, while the phase gate S allows obtaining complex probability amplitudes.

Although the set of gates we have implicitly defined $\{\,\text{CNOT}, \text{H}, \text{S}\,\}$ has all the required properties, it can be shown that it is not a universal set. In particular, the *Gottesman-Knill theorem* states that:

Theorem 7. *A quantum circuit containing only* CNOT, H *and* S *gates is efficiently simulated by a classical computer.*

We call the circuits generated by this set of gates the *Clifford group*, and the set $\{\,\text{CNOT}, \text{H}, \text{S}\,\}$ consequently takes the name of *Clifford group generator*. A set of universal gates must, therefore, generate more than the Clifford group, even though it has not been proven that this is a sufficient condition to be a universal set.

Examples of universal gate sets are:

- $\{\,\text{CNOT}, \text{H}, \text{T}\,\}$: replacing the phase gate with the T gate in the Clifford group generator allows us to obtain a universal set of gates;
- $\{\,\text{TOFFOLI}, \text{H}, \text{S}\,\}$: starting again from the Clifford group generator, we obtain a universal set by replacing the CNOT with the TOFFOLI gate;

14 Examples of these sets are: $\{\,\text{AND}, \text{NOT}\,\}$, $\{\,\text{OR}, \text{NOT}\,\}$, $\{\,\text{NAND}\,\}$ and $\{\,\text{TOFFOLI}\,\}$.

The concept of a universal set of gates in quantum computing is different from what we are used to with classical gates. In the classical case we can construct any truth table starting from a set of universal classical gates. The meaning of a universal set in quantum computation is instead described by the *Solovay-Kitaev theorem*:

Theorem 8. *Any universal gate set can approximate a quantum gate on* n *qubits to precision* ϵ *using* $\Theta(2^n \log^c(\frac{1}{\epsilon}))$ *gates for some constant* c.

The term 2^n should not be surprising because a unitary operator acting on n *qubits* (that is, the gate we are trying to approximate) is a matrix of dimension $2^n \times 2^n$. What is really relevant is the term $\log^c(\frac{1}{\epsilon})$, which tells us that we can reduce the approximation error at a cost that grows as a polynomial of a logarithm —this function is often called *polylog*— which is efficient.

2.2.3 Algorithms

We have discussed what quantum gates are and how they can be assembled to build a quantum circuit. We now see how this circuit must be constructed in order to be useful for solving a computational problem. In this section we introduce quantum oracle-based circuits and present the simplest oracle-based algorithm.

Quantum Oracles

An oracle is a Boolean function, that is, a function that acts on *qubits* and that can be composed with gates. In order to be translated into quantum gates we must ensure that the resulting gate is reversible. In general, it is possible to make a circuit reversible by using an extra *qubit* and applying the XOR gate between the output of the circuit and the extra *qubit*.

For example, if the function $f(x)$ acts on a single *qubit* we can construct a reversible gate $\mathbf{U}_f$ as shown in Figure 2.7. It is easy to see that regardless of the action of $f(x)$, which may or may not be reversible, knowing the outputs of the gate it is possible to reconstruct the inputs. We can rewrite the application of $\mathbf{U}_f$ to the *qubits* as:

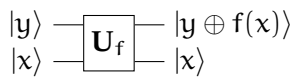


Figure 2.7: Quantum oracle gate

$$|x\rangle\,|y\rangle \xrightarrow{\mathbf{U}_f} |x\rangle\,|y \oplus f(x)\rangle$$

The extra *qubit* is called the *answer qubit* or *target qubit*, while $|x\rangle$ is called the *input qubit*. When we set $|y\rangle = |0\rangle$ we can read the result of $f(x)$ applied to $|x\rangle$ in the target *qubit*.

In reality, this is not the standard way of using a quantum oracle. What is done instead is to prepare the target *qubit* in the state $|-\rangle$ and then apply the gate $\mathbf{U}_f$. Quantum gates can, indeed, also be applied to superposed states, and in this case the result is a phase change of the *qubit* $|x\rangle$ while the target *qubit* remains unchanged. We can write the application of this gate as:

$$|x\rangle\,|-\rangle \xrightarrow{\mathbf{U}_f} \begin{cases} |x\rangle\,|-\rangle & \text{if } f(x) = 0 \\ -\,|x\rangle\,|-\rangle & \text{if } f(x) = 0 \end{cases} = (-1)^{f(x)}\,|x\rangle\,|-\rangle = (-1)^{f(x)}\,|x\rangle\,.$$

Where, in the last step, we neglect the value of the target *qubit* because it is not modified by the gate.

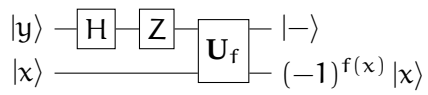


Figure 2.8: Phase oracle

Since, used in this way, the oracle applies a phase change to $|x\rangle$, it is called a *phase oracle*. Figure 2.8 shows the application of the phase oracle; in particular, it can be seen how it is possible to prepare $|y\rangle$ in the state $|-\rangle$ in order to then use $\mathbf{U}_f$ as a phase oracle[15].

Deutsch' algorithm

The Deutsch algorithm shows how to use the phase oracle to solve a computational problem. We will see that already with this first algorithm we obtain better performances compared to what we could achieve with a classical architecture. In this case, however, the performance gain is polynomial; therefore this is not one of those algorithms that suggest that quantum computers can efficiently solve a broader class of problems than classical computers[16].

The computational problem we want to solve is to determine the *parity* of two unknown bits b_1 and b_2. Determining parity means computing $b_1 \oplus b_2$. To obtain the result we are given access to an oracle $f(x)$ with $x \in \{0, 1\}$ which, when queried with the index of the unknown bit, returns the value of the bit, that is:

$$f(0) = b_0 \qquad f(1) = b_1 .$$

In a classical computation model we must query the oracle twice, obtain the values of the bits, and compute the result of the XOR between the two values.

Deutsch's algorithm allows us to obtain the parity value by querying the oracle only once. Figure 2.9 shows how the quantum circuit is implemented.

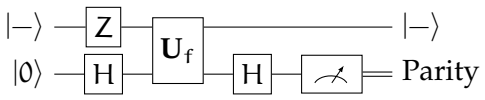


Figure 2.9: Deutsch' algorithm

Without developing all the algebraic steps, let us see what happens to the input *qubit* $|0\rangle$ as it passes through the quantum gates. The first quantum gate is the H gate; its application transforms $|0\rangle$ into

$$|0\rangle \xrightarrow{H} \frac{1}{\sqrt{2}} |0\rangle + |1\rangle = |+\rangle .$$

We now query the oracle with an input in a superposition of states. The result of the query is

$$\frac{1}{\sqrt{2}}(|0\rangle + |1\rangle) \xrightarrow{U_f} \frac{1}{\sqrt{2}}\Big[(-1)^{f_0} |0\rangle + (-1)^{f_1} |1\rangle\Big].$$

15 It is possible to obtain the same result by first applying the Paoli gate X and then the Hadamard gate H.

16 Showing these algorithms, including Shor's algorithm, is beyond the scope of this work.

Ignoring some steps we can simplify this expression as:

$$\begin{cases} (-1)^{b_0} |+\rangle & \text{if } b_0 = b_1 \\ (-1)^{b_0} |-\rangle & \text{if } b_0 \neq b_1 \end{cases}.$$

After the oracle, if $b_0 = b_1$ we obtain the state $|+\rangle$ up to a global phase; if $b_0 \neq b_1$ we obtain the state $|-\rangle$ up to a global phase. This phase does not modify the position of the *qubit* on the Bloch sphere, and we can already distinguish the two states with a measurement parallel to the x axis. Since, conventionally, measurements are performed with respect to the z axis, we apply the H gate once more obtaining

$$\begin{cases} (-1)^{b_0} |0\rangle & \text{if } b_0 = b_1 \\ (-1)^{b_0} |1\rangle & \text{if } b_0 \neq b_1 \end{cases}.$$

The act of measurement discards the global phase and allows us to distinguish the two cases with certainty: we obtain $|0\rangle$ if the unknown bits were equal (even parity), $|1\rangle$ if the unknown bits were different (odd parity). It is interesting to note that throughout the entire algorithm we never learn the value of the individual bits, but only whether they are equal or not.

Deutsch's algorithm allows us to halve the number of oracle queries, and it can be extended to an arbitrary number of bits whose parity we want to determine. This example concludes our discussion of the quantum gate-based paradigm. We will encounter this paradigm again in Chapter 7, where we will combine it with the concepts of quantum annealing that we now illustrate.